

Local Translation LLM — Hardware Assessment and Candidate Models

Prepared 2026-08-01 · Survey source: hpelitebook8g1i16, 2026-08-01T13:36:23-07:00

1. Purpose and scope

This assessment covers the hardware feasibility question only: which locally hosted language models the surveyed machine is capable of running, and at what throughput. It does not yet select a model for the translation workload, because the source and target languages have not been specified. Language pair is the single largest determinant of final model choice and can override every hardware consideration recorded here.

All performance figures below are estimates derived from memory bandwidth arithmetic, not measured benchmarks. They are intended for planning and shortlisting. Section 7 provides the measurement procedure that should replace them.

2. Machine specification as surveyed

Component	Value	Consequence for local inference
Model	HP EliteBook 8 G1i, 16 inch	Thin-and-light chassis; sustained thermal ceiling is modest
CPU	Intel Core Ultra 7 265U (Arrow Lake-U), 12 cores / 14 threads	U-series, ~15 W base. Adequate compute; not the bottleneck
SIMD	AVX, AVX2, AVX-VNNI, F16C. No AVX-512	AVX-VNNI accelerates int8 paths. AVX-512 absence costs little in llama.cpp
RAM	30 GiB total, 27 GiB available at survey time	Comfortably runs models up to roughly 24 GB on disk
Swap	512 MiB swapfile	Too small. See Section 6, Action 1
GPU	Intel Arrow Lake-U integrated graphics (8086:7d41). No discrete GPU	Unified memory: shares the same RAM and the same memory bus
NPU	Present (marketed AI PC). Not detected by this survey	Not usable for LLM inference today. See Section 5
Storage	KIOXIA NVMe 476.9 GB, 353 GB free	Ample. Not a constraint
OS	Ubuntu 26.04 LTS, kernel 7.0.0-27-generic	Current; good driver support for Intel graphics
Installed	Ollama, Docker, Python 3. Model qwen3.6 (23 GB) already pulled	Substantial head start. See Section 4

3. The binding constraint is memory bandwidth

The most important finding is that this machine is memory-bandwidth-bound, not compute-bound. During text generation the inference engine must read the model weights from RAM once per token produced. Token throughput is therefore governed by how fast weights can be streamed out of memory, and the processor spends most of its time waiting.

The practical rule follows directly:

$$\text{tokens per second} \approx \text{effective memory bandwidth} \div \text{bytes read per token}$$

Arrow Lake-U provides a 128-bit dual-channel memory interface. Depending on which memory this unit shipped with, theoretical bandwidth falls between roughly 90 GB/s (DDR5-5600 SODIMM) and 136 GB/s (LPDDR5x-8533 soldered). Real-world efficiency for inference typically lands between 50 and 65 percent of theoretical. The estimates in Section 4 assume approximately 65 GB/s effective. The survey could not read the memory type because it ran without elevated privileges; Section 7 gives the command to resolve this.

3.1 Why this makes Mixture-of-Experts models decisive

A dense model reads every parameter for every token. A Mixture-of-Experts (MoE) model holds all parameters in memory but activates only a small subset per token, so it reads far less. On bandwidth-bound hardware this is not a marginal optimization; it is the difference between a usable tool and an unusable one.

A 27B dense model and a 35B MoE model with 3B active parameters occupy comparable memory, yet the MoE model generates text roughly six to eight times faster on this class of machine. Every recommendation below follows from this fact.

3.2 The integrated GPU does not solve it

Offloading to the integrated GPU is worth doing, but it should be understood correctly. Because the iGPU shares the same physical memory bus as the CPU, offloading does not increase the available bandwidth. It therefore delivers little improvement to token generation.

It does materially accelerate prompt processing, which is compute-bound rather than bandwidth-bound. For translation work this matters more than it would for chat, because each request feeds a long source passage into the model before any output is produced. Expect a meaningful reduction in time-to-first-token and only a modest change in generation speed.

4. Candidate models

The list below is filtered by hardware feasibility alone. Ordering within each tier runs from most to least recommended for this machine.

4.1 Tier 1 — recommended: MoE models

Model	Size on disk	Est. speed	Notes
qwen3.6:35b-a3b	~24 GB	~20–30 tok/s	Already installed. 3B active parameters, 256K context, Apache 2.0. Best quality-per-second available on this hardware
qwen3.5:30b-a3b or similar	~18 GB	~25–35 tok/s	Previous generation. Lighter memory footprint leaves more headroom for context

4.2 Tier 2 — fast dense models for bulk or draft passes

Model	Size on disk	Est. speed	Notes
gemma4:12b (Q4)	~8 GB	~7–9 tok/s	Gemma lineage carries very broad language coverage. Strong candidate for European pairs
qwen3.5:8b (Q4)	~5 GB	~12–15 tok/s	Leaves large headroom for long context windows
gemma4:e4b / qwen3.5:4b	~3–4 GB	~25–30 tok/s	Fast enough for interactive use. Quality drops noticeably on nuanced text

4.3 Tier 3 — technically runnable but poorly matched

Model	Size on disk	Est. speed	Notes
qwen3.6:27b (dense)	~17 GB	~3–4 tok/s	Fits in memory but every parameter is read per token. Roughly a minute per paragraph
gemma4:26b / 31b (dense)	~17–20 GB	~3–4 tok/s	Same limitation. Consider only for a final polish pass on short, high-value text

These are listed for completeness because the original question asked what the machine can theoretically run. They fit in memory. They are not recommended as a primary translation engine.

4.4 Tier 4 — dedicated machine translation models

These are purpose-built translation systems rather than general language models. They are dramatically smaller and faster, and on high-resource language pairs they are often more accurate than a general model many times their size. Two caveats apply: most operate at sentence level and will not carry terminology or register consistently across a long document, and they are generally not distributed through Ollama, so they require a separate runtime such as CTranslate2 or Hugging Face Transformers.

Model	Size	Coverage	Notes
NLLB-200 (1.3B / 3.3B)	~3–7 GB	200 languages	Meta. The reference option for low-resource and less common languages
Opus-MT (Helsinki-NLP)	~300 MB per direction	One pair each	Extremely fast. Best where the pair is fixed and volume is high

Availability and current versions of these two families were not verified during this assessment and should be confirmed before committing to either.

5. What is not available

- The NPU cannot be used. Intel Core Ultra processors include a neural processing unit, but it is not supported for Ollama inference. Support exists in other Intel toolchains but has not reached the Ollama path. The NPU should be treated as unavailable for this project.
- CUDA and any NVIDIA-dependent tooling. The nvidia-smi binary is present on the system but non-functional, which suggests a leftover package rather than hardware. Removing it would prevent confusion in future surveys.
- Models above roughly 26 GB on disk. Anything larger leaves insufficient room for the key-value cache and the desktop session.

6. Actions required before production use

Action 1 — Increase swap (do this first)

A 512 MiB swapfile is a genuine risk. The installed model occupies 23 GB against 27 GB available, leaving roughly 4 GB for the key-value cache, the desktop, and a browser. If that margin is exceeded, the kernel has almost no swap to fall back on and the out-of-memory killer terminates the process abruptly. Swap here is a safety net rather than a performance measure; inference should never actually touch it.

```
sudo swapoff /swapfile
sudo fallocate -l 8G /swapfile
sudo chmod 600 /swapfile
sudo mkswap /swapfile && sudo swapon /swapfile
```

```
free -h # confirm 8.0Gi swap
```

Action 2 — Constrain the key-value cache

Long source documents produce a large key-value cache, which is what will consume the remaining headroom. Quantizing the cache reduces this substantially at negligible quality cost.

```
# add to ~/.profile or a systemd override for the ollama service
export OLLAMA_KV_CACHE_TYPE=q8_0
export OLLAMA_FLASH_ATTENTION=1
export OLLAMA_KEEP_ALIVE=30m # avoid reloading 23 GB repeatedly
```

Action 3 — Run on mains power

The survey recorded the battery as not charging. A 15 W base-TDP processor throttles hard under sustained inference load on battery. Connect mains power and confirm the platform profile is not set to power-saver before any timing measurement.

```
cat /sys/firmware/acpi/platform_profile
powerprofilesctl list
```

7. Commands to run next

Two gaps in the survey should be closed, and one baseline measurement taken.

Resolve the memory type, which sets the true bandwidth ceiling and reveals whether upgrade is possible:

```
sudo dmidecode -t memory | grep -E 'Size:|Type:|Speed:|Locator:|Form Factor:'
```

Confirm which qwen3.6 variant is installed. A 23 GB footprint indicates the 35B-A3B MoE build, which would be the ideal outcome, but this should be verified rather than inferred:

```
ollama show qwen3.6
ollama list --format json 2>/dev/null || ollama list
```

Take a throughput baseline to replace the estimates in Section 4:

```
ollama run qwen3.6 --verbose "Translate the following into English. Output only
the translation, preserving paragraph breaks:" < sample_source.txt

# --verbose prints prompt eval rate (prefill) and eval rate (generation)
# Record both. Prefill governs long-document latency; eval governs output speed.
```

Optionally, evaluate integrated-GPU offload. Mainline Ollama does not accelerate Intel integrated graphics; this requires either a SYCL-enabled build or Intel's IPEX-LLM fork, both most easily obtained as containers. Docker is already installed. Given the bandwidth analysis in Section 3.2, this should be treated as a prompt-processing optimization and deferred until the CPU baseline is measured.

```
ls -la /dev/dri/ # confirm render node is exposed
sudo usermod -aG render,video $USER # then log out and back in
```

8. Open question blocking final selection

The hardware question is now answered. Model selection is not, and turns primarily on one input.

- Source and target languages. General-purpose models degrade sharply outside high-resource pairs, and evaluations have found some general models producing near-unusable output on the lowest-resource targets. If a mid- or low-resource language is involved, NLLB-200 or a language-specific model may outrank everything in Tier 1 regardless of the hardware advantage.

- Source document format. Scanned PDFs require an additional offline optical character recognition stage, which is a separate build with its own accuracy characteristics.
- Quality bar and volume. Draft comprehension and publication-grade output imply different architectures, as does a few pages per month versus a few hundred.
- Whether the confidentiality requirement is contractual or regulatory rather than a personal preference. If formal, model licensing and provenance become selection criteria in their own right, independent of quality.

9. Provisional conclusion

The machine is well suited to this task, more so than the absence of a discrete graphics card would suggest. Thirty gigabytes of RAM places it above the threshold where the most efficient current open-weight architecture becomes viable, and the model that best exploits that architecture appears to be installed already.

Subject to confirmation of the installed variant and to the language-pair question, the provisional recommendation is to build the translation workflow on the Qwen 3.6 35B-A3B MoE model, retain a small dense model such as Gemma 4 12B for rapid draft passes and comparison, and evaluate a dedicated machine translation system as a cross-check on terminology if the language pair proves to be low-resource.